Χρυσόστομος Κουμπάνης 4325

Hi, in this assignment i used Transfer Learning to classify images from the CIFAR-10 dataset. Unlike Assignment 1 where i built a CNN from scratch, here i used a pre-trained VGG16 model (trained on ImageNet) to extract features. The goal was to feed these features into Support Vector Machines (SVM) and compare them against other models like k-NN and MLP.

Data Preprocessing & Pipeline:

Before training, i processed the data through a specific pipeline to ensure compatibility with VGG16 and optimal performance for the SVM.

The raw CIFAR-10 images are in RGB format.

I used the 'preprocess_input' function from Keras, which converts the images from RGB to BGR and zero-centers each color channel with respect to the ImageNet dataset mean. This step is crucial because VGG16 was trained on data processed in this specific way.

Feature Extraction:

I loaded VGG16 (excluding the top classification layers) and used it as a "frozen" feature extractor.

The output was a feature vector of 512 numbers for every image.

Standardization & PCA:

I applied `StandardScaler` to normalize the features (mean=0, variance=1).

I then used Principal Component Analysis (PCA) to reduce dimensionality, retaining 90% of the variance. This reduced the feature space from 512 to approximately 329 dimensions, making the SVM training faster and less prone to overfitting.

SVM Training & Testing:

I extensively tested the SVM with three kernels: **Linear**, **Radial Basis Function** (**RBF**), and **Polynomial**. I conducted these tests to tune the model.

Test 1: Small Subset (5,000 Training Images)

I started with a small subset to get a baseline.

Model	Training Time	Training Accuracy	Testing Accuracy	Observation
Linear Kernel	7.44 seconds	97.08%	54.4%	Overfitting
RBF Kernel	2.72 seconds	84.62%	59.7%	Best Performer
Polynomial Kernel	3.78 seconds	66.12%	36.0%	Underfitting

The **RBF kernel** immediately showed it could handle the non-linear relationships better than the Linear kernel.

Test 2: Increased Dataset (15,000 Training Images) and increased C from 1.0 to 10.0 for the polynomial kernel to allow fewer errors in training

I increased the data to see how the models scaled.

Model	Training Time	Training Accuracy	Testing Accuracy	Observation
Linear Kernel	3278 seconds (~55 min)	79.17%	59.85%	Optimizer struggled massively (Not linearly separable)
RBF Kernel	53.56 seconds	98.17%	63.05%	Best Performer Again
Polynomial Kernel	65.0 seconds	93.77%	60.15%	Overfitting

- It took ~55 minutes to run the linear kernel, that means that the optimizer struggled massively to converge, proving that CIFAR-10 data is not linearly separable.
- The polynomial kernel performed faster than the Linear kernel but less accurate than the RBF kernel. It performed decently but offers no advantage over RBF for this specific dataset.

Test 3: Increased C from 10.0 to 100.0 to see if the polynomial kernel would perform better

I didnt include the linear kernel, because it would take a lot of time without any improvement, also the increase in the C hyperparameter wouldnt affect it.

Model	Training Time	Training Accuracy	Testing Accuracy	Observation
RBF Kernel	54.31 seconds	100.0%	60.7%	Overfitting (Memorized noise)
Polynomial Kernel	61.89 seconds	99.57%	59.75%	Overfitting even more than before

The reason why the polynomial kernel didnt perform better is because when C is equal to 100, the SVM is very strict and memorized every little detail and noise in the 15000 training set and got **99.57%** accuracy on training.Thats why on the test it performed worse.

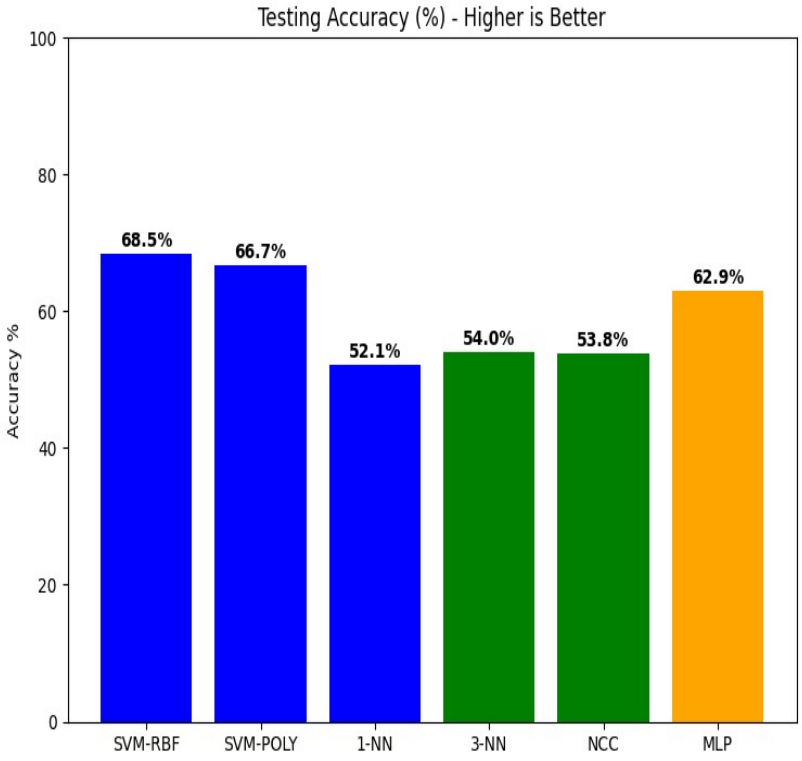
Final Test (Full Dataset):

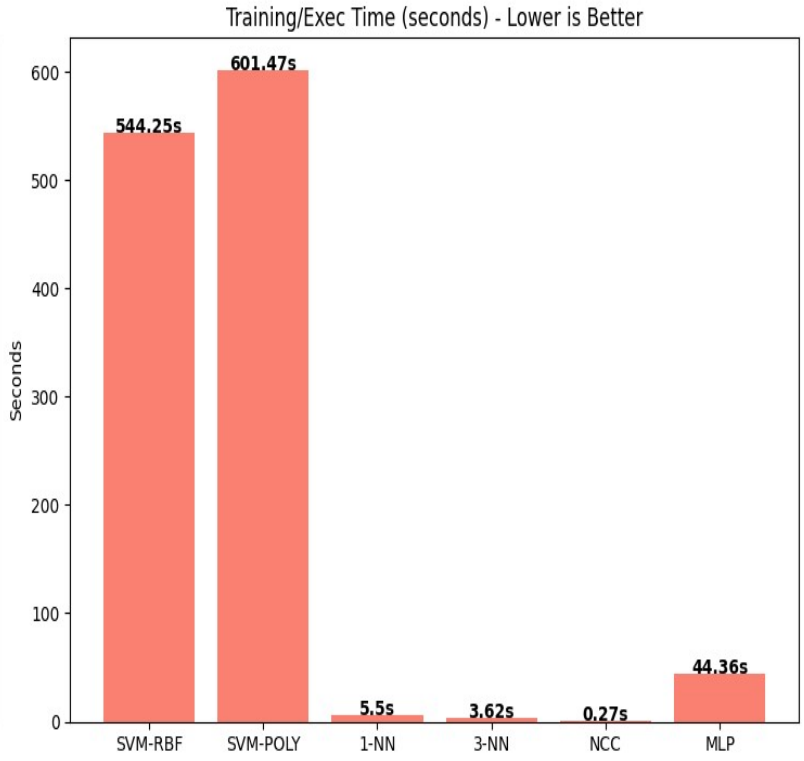
For the final run, I used the full image dataset. Also i excluded the Linear Kernel from this run because, based on Test 2, it was projected to take over 15 hours to converge.

I compared the optimal SVMs (RBF/Poly with C=10) against k-NN, Nearest Class Centroid (NCC), and an MLP with Hinge Loss.

Model	Testing Accuracy	Training Time	Analysis
SVM (RBF)	68.50%	~544 seconds	The Winner. Best generalization.
SVM (Poly)	66.74%	~601 seconds	Good, but slightly slower/less accurate.
MLP (Hinge)	62.93%	~44 seconds	Fastest Learner. Good trade-off.
3-NN	54.05%	~3.6 seconds	Too simple for complex images.
NCC	53.84%	~0.2 seconds	Too simple for complex images.
1-NN	52.11%	~5.5 seconds	Too simple for complex images.

Visual Comparison:





Comparison with Assignment 1 (Custom CNN):

In my previous assignment, i built a custom CNN trained from scratch that achieved 84% accuracy. In this assignment, the best result was 68.5% (SVM-RBF).

Why the drop in performance?

1) Resolution Mismatch: VGG16 is designed for high-resolution ImageNet images. When it is fed tiny CIFAR-10 images the network loses spatial details too quickly.

2)Frozen Weights: In Assignment 1, the CNN adapted specifically to CIFAR-10. Here, VGG16 used generic filters, which limits its ability to pick up fine-grained details in low-resolution photos.

Conclusion:

This assignment demonstrated that SVMs with RBF kernels are powerful tools for classification when combined with deep learning features, significantly outperforming simple methods like k-NN. However, for low-resolution image tasks, a specialized CNN trained from scratch (Assignment 1) remains superior to a frozen Transfer Learning model.